

Face Detection Using OpenCV

A Command-Line Face Detection Tool

Course: Computer Vision

Project Report

Submitted as part of the Flipped Course Evaluation

1. Introduction

Face detection is one of the foundational tasks in computer vision, underlying applications ranging from photo organization to security and human-computer interaction. This project implements a command-line tool that detects human faces in images, video files, and live webcam streams using OpenCV, offering a choice between a classical Haar Cascade classifier and a deep-learning based (DNN/SSD) detector. The tool draws bounding boxes around every detected face and reports a running face count, saving the annotated result to disk.

2. Problem Statement

Manually reviewing images or video footage to locate human faces is slow and error-prone. There is a need for a lightweight, easy-to-run tool that can automatically locate faces in an image, a video file, or a live camera feed, and clearly mark where they were found, without requiring a GUI or a heavy machine-learning framework.

Scope: the project detects the presence and location of faces (bounding boxes only). Face recognition (identifying who a face belongs to), age/emotion estimation, and training a new detection model from scratch are explicitly out of scope.

Target users: students learning computer vision fundamentals, developers who need a quick scriptable face-detection step for a larger pipeline, and anyone wanting to batch-annotate photos or videos with face bounding boxes.

3. Functional Requirements

- **Image Processing Module:** load an image file, run face detection on it, draw bounding boxes and a face count, and save the annotated result.
- **Video/Webcam Processing Module:** read frames from a video file or a live webcam, run face detection per frame (with configurable frame-skip), and write an annotated output video.
- **Face Detection Module:** provide a common interface over two interchangeable detection backends — a Haar Cascade classifier and a DNN/SSD deep-learning detector — selectable via a command-line flag.
- **CLI Interface:** a single entry point (main.py) exposing all three modes (image / video / webcam) with clear, self-documenting arguments (--help).

4. Non-Functional Requirements

Requirement	How it is addressed
Performance	A --frame-skip option lets the detector run every N frames instead of every single frame during video/webcam processing, trading accuracy for speed on slower machines.
Usability	A simple, self-documenting CLI with --help output, sensible defaults (Haar method, auto-named outputs), and consistent flags across all three modes.
Reliability	Input files, cascade files, and DNN model files are validated before use; the tool raises clear, specific errors (FileNotFoundError, ValueError, IOError) instead of crashing unpredictably.
Maintainability	A modular package layout with a single-responsibility per file, a common FaceDetector abstract interface, and one central config.py for every tunable value.

Error Handling Strategy	All expected failure modes (missing file, unreadable image, missing model weights, bad camera index, unknown detection method) are caught at the CLI boundary and reported to the user with an actionable message.
Logging / Monitoring	Every run logs its input source, detection method, face count, and elapsed time to both the console and a persistent logs/face_detection.log file via a shared logger.

5. System Architecture

The system follows a layered, modular architecture. A CLI layer parses user input and delegates to either the Image Processor or the Video Processor, both of which depend only on the abstract FaceDetector interface, not on a specific detection algorithm. This means a new detector (e.g. a future MediaPipe or MTCNN backend) could be added without touching the processing or CLI layers.

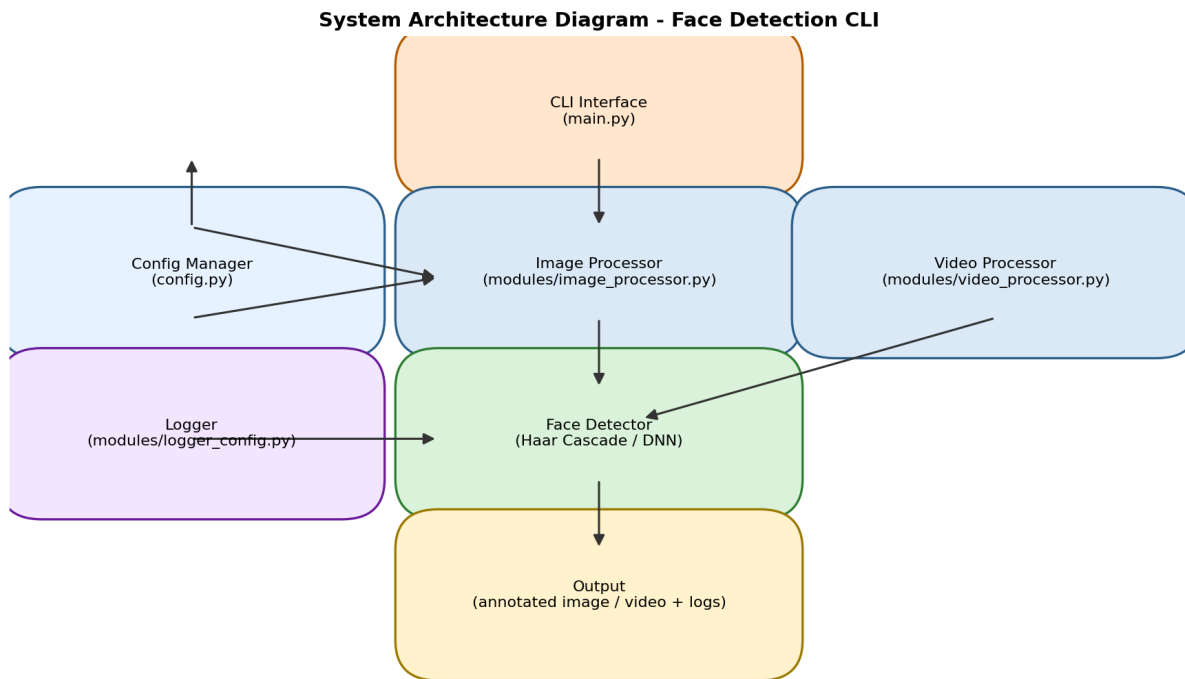


Figure 1: System Architecture Diagram

6. Design Diagrams

6.1 Use Case Diagram

Use Case Diagram - Face Detection CLI

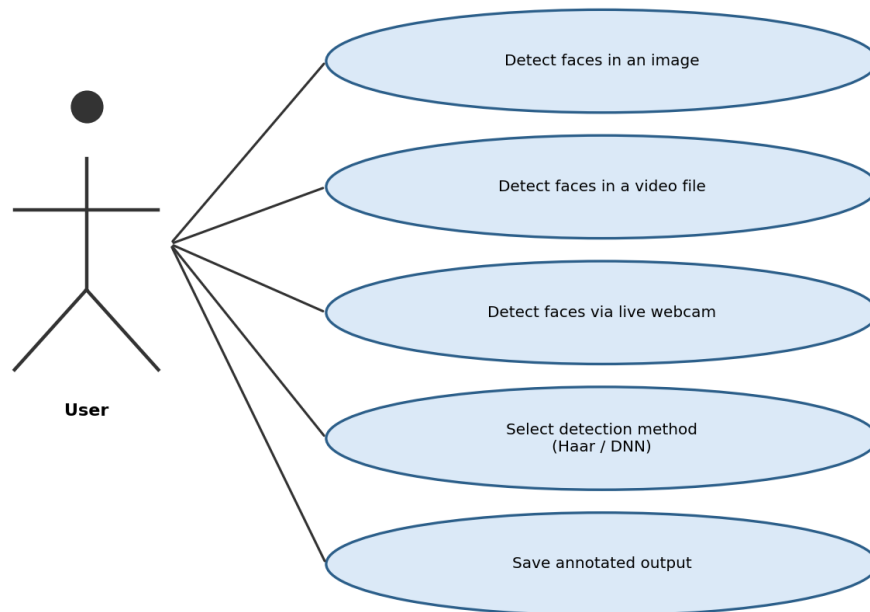


Figure 2: Use Case Diagram

6.2 Workflow / Process Flow Diagram

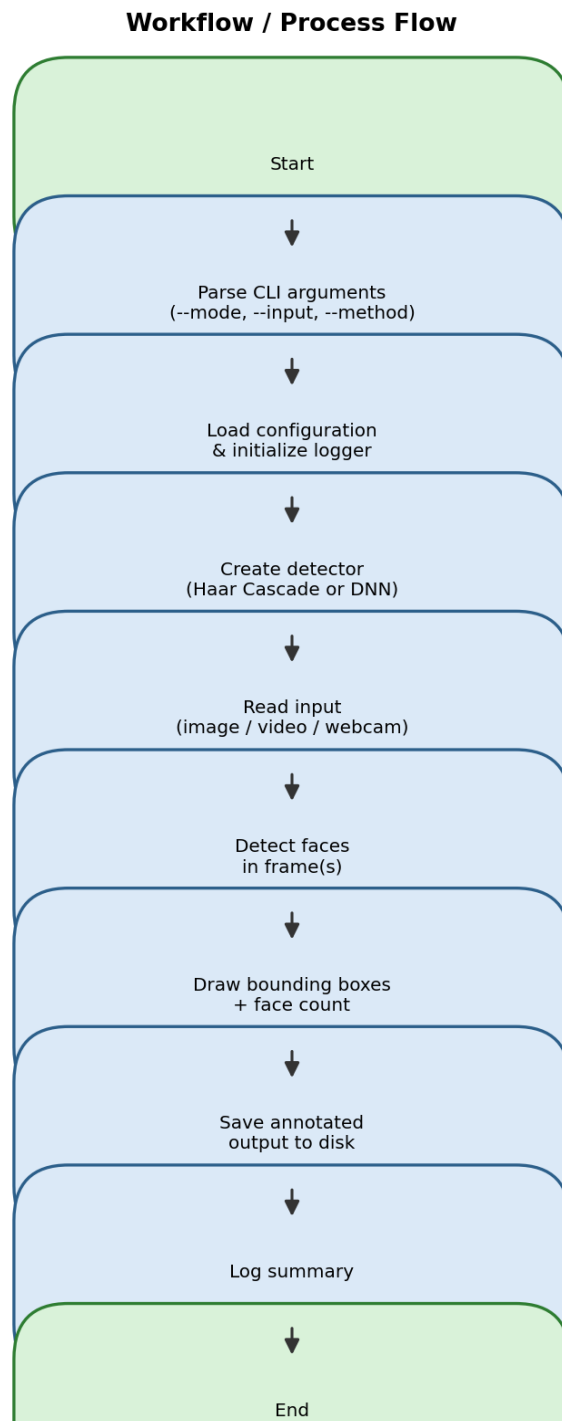


Figure 3: Workflow / Process Flow Diagram

6.3 Sequence Diagram (Image Mode)

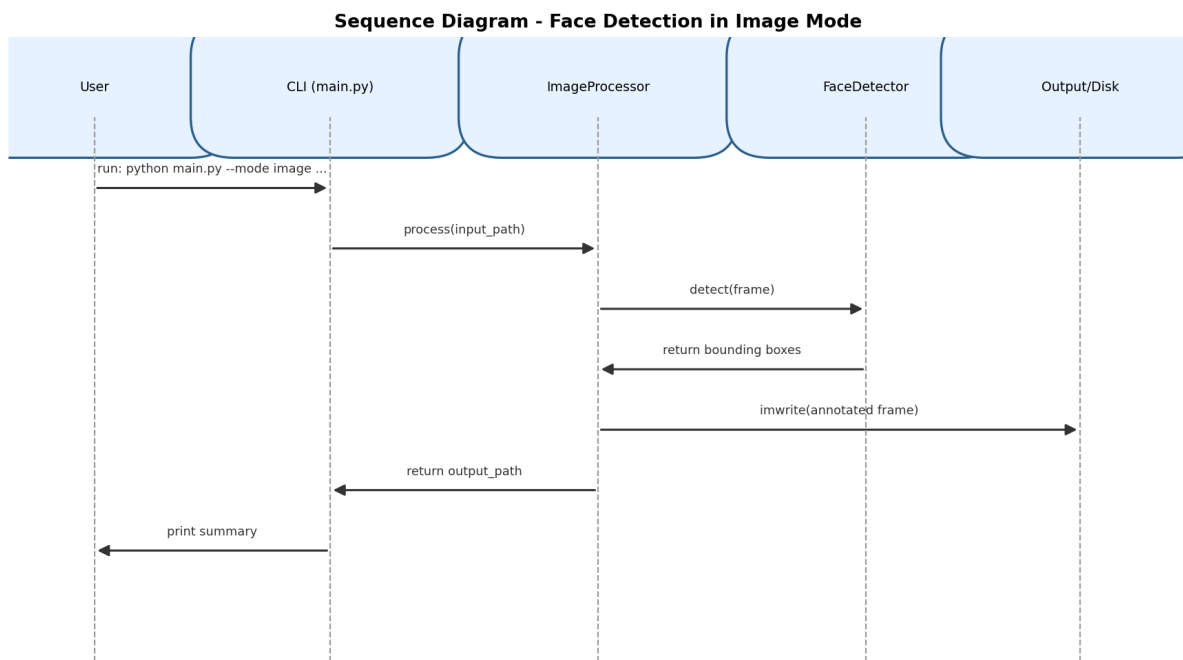


Figure 4: Sequence Diagram

6.4 Class Diagram

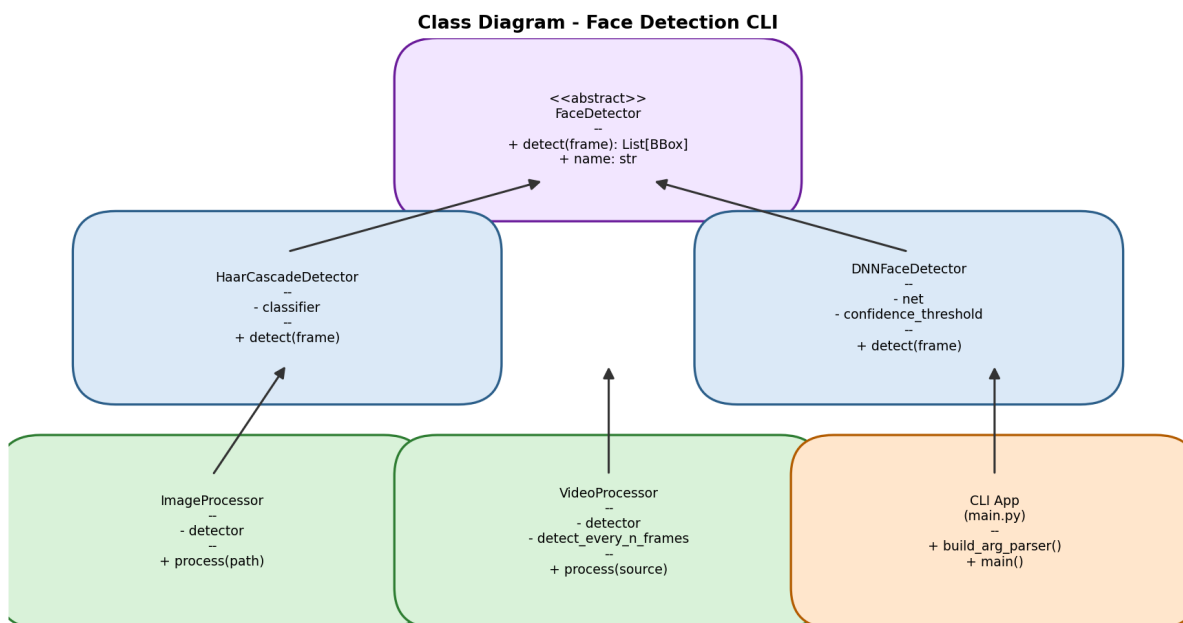


Figure 5: Class Diagram

6.5 Database / Storage Design

Not applicable. This project does not persist data in a database; it reads image/video files from disk and writes annotated image/video files back to disk (see the output/ folder), so no ER diagram or schema is required.

7. Design Decisions & Rationale

- **Abstract FaceDetector base class:** chosen so the Image/Video processors never need to know which algorithm is running underneath — new detectors can be plugged in via the `create_detector()` factory alone.
- **Haar Cascade as the default backend:** it ships inside OpenCV itself, so a fresh checkout of the project works immediately with zero downloads, which matters for a course project that must be easy for an evaluator to run.
- **DNN backend kept optional:** the more accurate SSD-based detector needs external model weight files; rather than bundling large binaries in the Git repository, the project documents exactly where to download them.
- **Frame-skip parameter:** running a detector on every frame of a video/webcam stream is the main performance bottleneck; making the skip interval configurable lets the same code scale from a fast desktop to a slower machine.
- **Centralized config.py:** keeping thresholds, paths, and drawing settings in one file avoids magic numbers scattered across modules and makes tuning the detector's sensitivity a one-line change.

8. Implementation Details

The project is implemented in Python 3 using OpenCV (cv2) and NumPy. It is organized into a modules/ package with one file per responsibility:

- **modules/face_detector.py** – defines the FaceDetector abstract base class and two implementations: HaarCascadeDetector (cv2.CascadeClassifier) and DNNFaceDetector (cv2.dnn, Caffe SSD model), plus a `create_detector()` factory.
- **modules/image_processor.py** – loads a single image with `cv2.imread`, validates it, runs the detector, draws results, and saves the annotated image.
- **modules/video_processor.py** – wraps `cv2.VideoCapture` / `cv2.VideoWriter` to process a video file or webcam stream frame by frame, supporting frame-skipping and an optional live preview window.
- **modules/utils.py** – shared drawing helpers (bounding boxes, face-count overlay) and a summary-string formatter used for logging.
- **modules/logger_config.py** – configures one shared logger writing to both stdout and a rotating log file.
- **config.py** – all paths, detection thresholds, and drawing constants.
- **main.py** – argparse-based CLI that wires the above modules together for the image / video / webcam modes.

9. Screenshots / Results

Below is an illustrative preview of the tool's annotated output style: green bounding boxes are drawn around each detected face, labeled 'Face', with a running total shown in the corner. When run against a real photo or webcam feed, the same drawing logic (modules/utils.py: draw_detections and annotate_face_count) produces boxes around the actual detected faces.

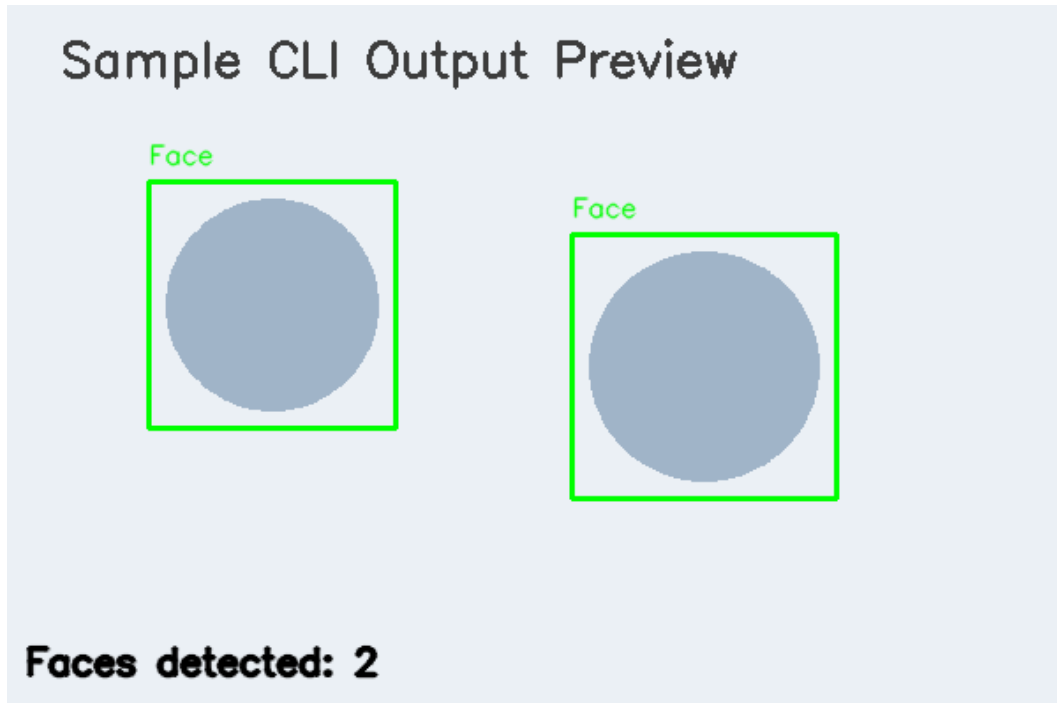


Figure 6: Illustrative annotated-output preview

A sample CLI run (Haar Cascade, image mode) on a real photo produces console output similar to:

```
Done. Annotated image saved to: output/photo_detected.jpg
```

10. Testing Approach

Unit tests (tests/test_face_detector.py, tests/test_image_processor.py) use pytest and synthetically generated images/frames so they run without any external dataset. They verify:

- The Haar Cascade classifier loads successfully and reports 0 faces on a blank frame.
- The detector factory returns the correct backend and rejects unknown method names with a `ValueError`.
- Loading a missing image raises `FileNotFoundError`, and a corrupted/non-image file raises `ValueError`, instead of failing silently.
- A successful run of `ImageProcessor.process()` produces a valid, readable output image file on disk.

In addition, the full CLI (main.py) was manually smoke-tested end-to-end in image mode to confirm the argument parser, detector, processor, and logger all integrate correctly.

11. Challenges Faced

- Balancing detection accuracy against speed for video/webcam input — solved with the configurable `--frame-skip` option rather than a fixed frame rate.

- Keeping the DNN backend genuinely optional (since its model files are large binaries that shouldn't live in Git) while still giving a clear, actionable error message when someone tries to use `--method dnn` without downloading them.
- Designing a detector interface general enough that Haar Cascade and DNN/SSD — two very different underlying algorithms — can be swapped in without any change to the calling code.

12. Learnings & Key Takeaways

This project reinforced how a clean abstraction (the FaceDetector interface) pays off immediately even in a small codebase: the image and video pipelines were written once and never needed to change when a second detection backend was added. It also highlighted the practical trade-off between classical computer-vision techniques (Haar Cascades: fast, no training data, weaker on difficult poses/lighting) and deep-learning detectors (DNN/SSD: more robust, but heavier and dependent on pre-trained weights).

13. Future Enhancements

- Add a third backend using MediaPipe or a modern MTCNN model for better accuracy on small/occluded faces.
- Batch-process an entire folder of images in one command.
- Optional face blurring/anonymization mode for privacy-preserving use cases.
- Export detection results (bounding boxes, confidence, timestamps) as a structured CSV/JSON report alongside the annotated media.

14. References

- OpenCV Documentation – Cascade Classifier: <https://docs.opencv.org/>
- OpenCV DNN Face Detector (SSD/ResNet, Caffe model) – OpenCV GitHub samples repository.
- Viola, P. and Jones, M. (2001). Rapid Object Detection using a Boosted Cascade of Simple Features.
- Python argparse and logging standard library documentation.